

TP4 - Enrichissement des passerelles de paiement e-commerce

Decorator Pattern

Filière / module : Génie logiciel - Design Patterns & Frameworks
Niveau : 2e année cycle ingénieur - Travail individuel ou en binôme
Durée : 2 heures

Objectif du TP. Le TP1 a permis d'unifier plusieurs fournisseurs externes de paiement derrière une même abstraction `PaymentGateway` grâce au patron `Adapter`.

Le TP2 a ensuite permis de choisir dynamiquement la meilleure passerelle de paiement selon le contexte de la commande grâce au patron `Strategy`.

Le TP3 a isolé la création des catalogues de passerelles selon la boutique grâce au patron `Factory Method`.

Le TP4 traite un nouveau point variable : **comment ajouter des responsabilités autour d'une passerelle sans modifier les adaptateurs existants ?**

L'objectif est de mettre en œuvre le patron `Decorator` afin d'ajouter logging, frais de service, contrôle anti-fraude, cashback ou retry tout en conservant le contrat `PaymentGateway`.

Dans cette conception, les quatre patrons sont complémentaires :

- `Adapter` rend compatibles les fournisseurs externes avec le contrat `PaymentGateway`.
- `Strategy` choisit quelle passerelle utiliser selon une politique métier interchangeable.
- `Factory Method` crée la famille de passerelles disponible selon le type de boutique.
- `Decorator` enrichit une passerelle déjà créée sans changer son interface.

Positionnement pédagogique

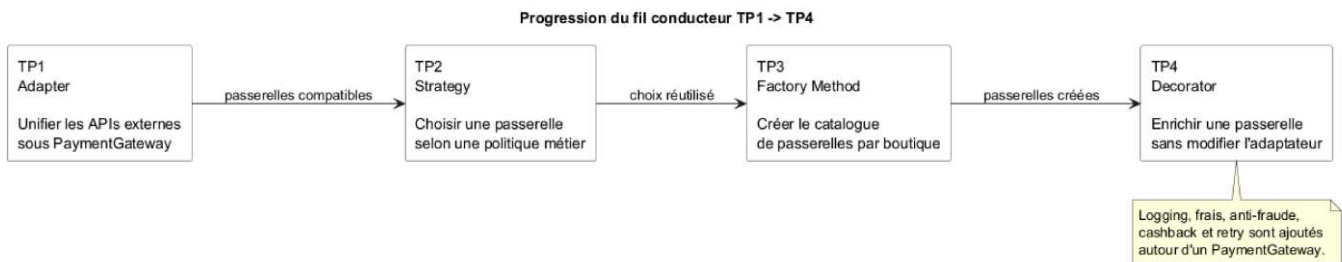
Ce TP prolonge directement le problème métier des TP1, TP2 et TP3 sur le même domaine de paiement e-commerce.

Après avoir stabilisé l'intégration technique des fournisseurs, la sélection métier d'une passerelle et la

création des catalogues par boutique, on cherche maintenant à ajouter des comportements optionnels autour des passerelles.

Le problème métier reste le même : traiter un paiement dans un checkout e-commerce.

Le point de variation change : il ne s'agit plus d'adapter une API externe, de choisir une passerelle ou de créer un catalogue, mais d'enrichir dynamiquement une passerelle existante.



Source PlantUML : `../diagrams/figure4_progression_tp1_tp4.puml`

Un guide d'exploration est fourni dans `docs/GUIDE_EXPLORATION_DECORATOR.md`.

Il explique le rôle du patron **Decorator**, les erreurs de conception à éviter et sa complémentarité avec les patrons déjà utilisés.

1. Contexte professionnel

Vous intervenez sur une plateforme e-commerce qui propose plusieurs boutiques : Maroc, Europe, Premium et Afrique.

Grâce au TP3, chaque boutique sait créer son propre catalogue de passerelles.

Le métier demande maintenant d'ajouter des traitements transverses autour des paiements :

- journaliser les tentatives de paiement ;
- ajouter des frais de service propres à certaines boutiques ;
- bloquer les paiements suspects ;
- accorder un cashback aux clients premium ;
- relancer automatiquement un paiement en cas d'échec temporaire.

Ces traitements ne changent pas l'API externe de PayPal, du virement bancaire ou du wallet mobile. Ils viennent se placer autour d'une passerelle déjà existante.

2. Problème à résoudre

Une première solution consisterait à modifier directement les adaptateurs :

```

public class PaypalAdapter implements PaymentGateway {
    public PaymentResult pay(CheckoutContext context) {
        // logging
        // anti-fraude
        // frais de service
        // appel PayPal
        // cashback
    }
}

```

Cette solution est fragile pour trois raisons.

Premièrement, elle mélange deux responsabilités : adapter une API externe et appliquer des règles métier transverses.

Deuxièmement, elle duplique les mêmes traitements dans plusieurs adaptateurs.

Si le logging change, il faudra modifier **PaypalAdapter**, **BankTransferAdapter** et **MobileWalletAdapter**.

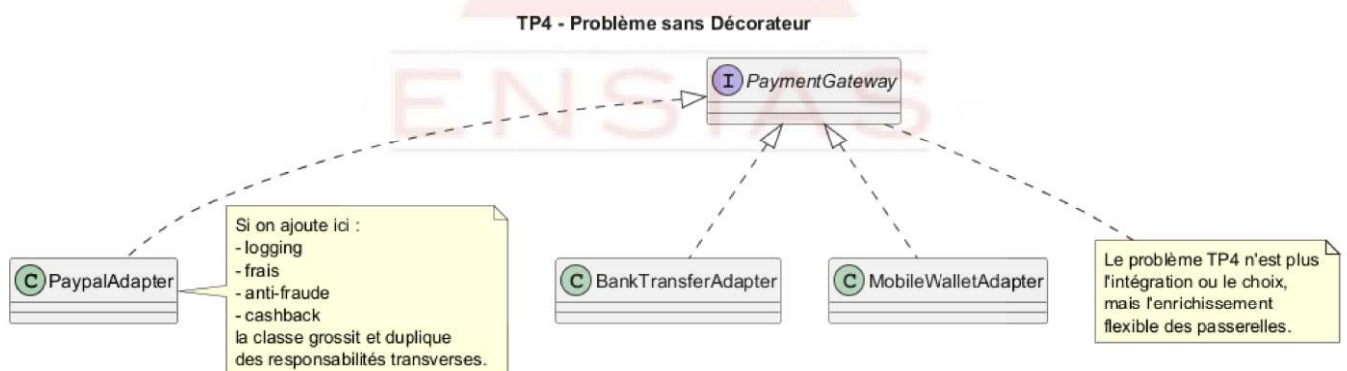
Troisièmement, elle rend les combinaisons difficiles.

Une boutique peut vouloir logging + frais, une autre logging + retry, une autre anti-fraude + cashback.

Multiplier les sous-classes ou les conditions rendrait le code peu lisible.

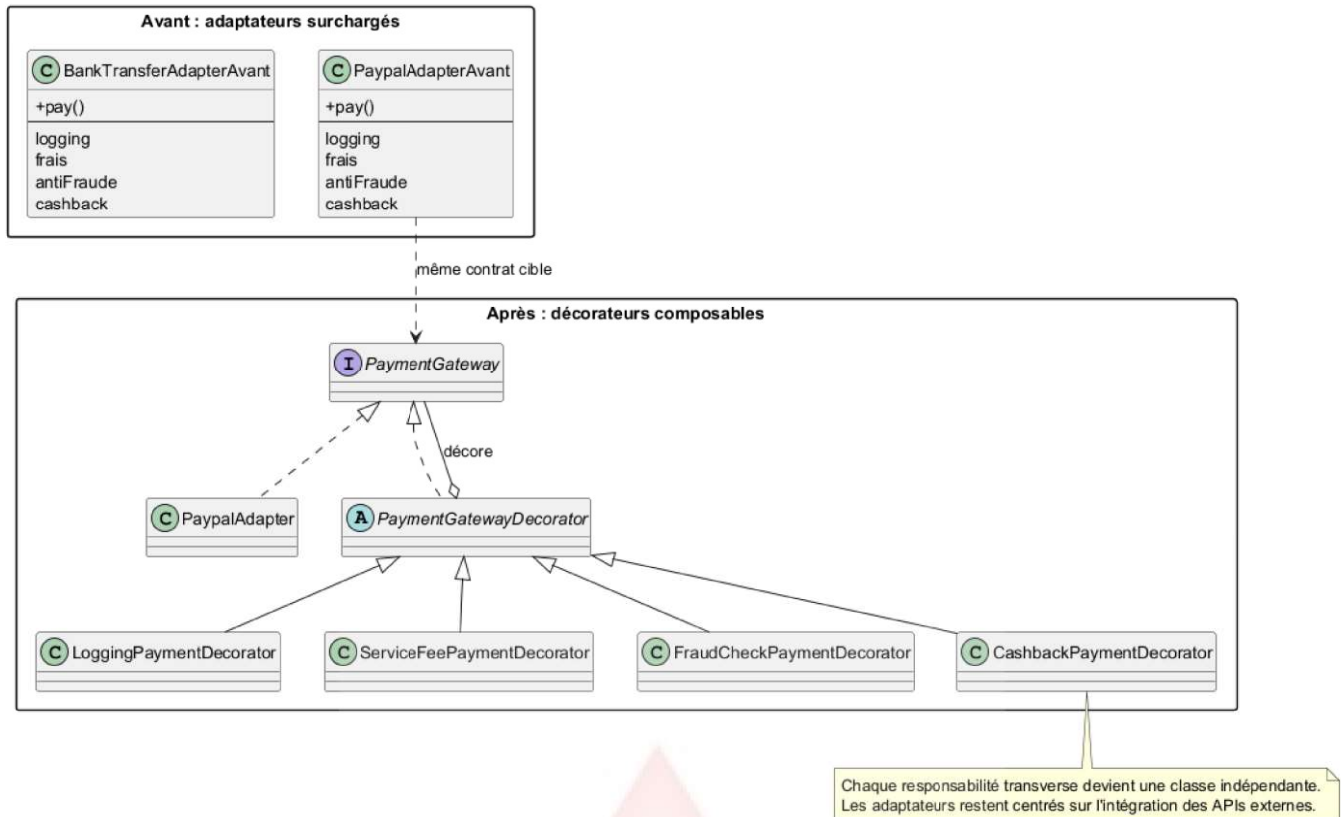
Le patron **Decorator** permet de résoudre ce problème en enveloppant une passerelle par un objet qui implémente la même interface.

Le service de checkout continue donc à manipuler uniquement des **PaymentGateway**.



Source PlantUML : `../diagrams/figure1_problem_without_decorator.puml`

Avant / après : déplacer les responsabilités transverses

Source PlantUML : `../diagrams/figure5_before_after_decorator.puml`

3. Structure cible

Le composant commun reste l'interface déjà utilisée depuis les TP's précédents :

```

public interface PaymentGateway {
    boolean supports(CheckoutContext context);
    double estimateFee(CheckoutContext context);
    int estimateConfirmationDelayMinutes(CheckoutContext context);
    int getSecurityScore();
    PaymentResult pay(CheckoutContext context);
    String getProviderName();
}
  
```

Le décorateur abstrait conserve une référence vers une passerelle décorée :

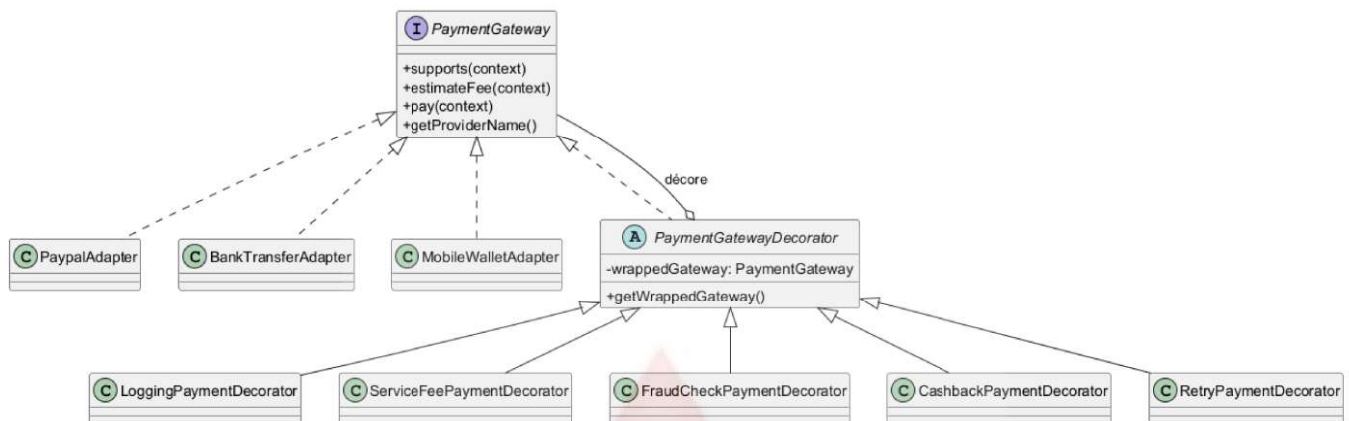
```

public abstract class PaymentGatewayDecorator implements PaymentGateway {
    private final PaymentGateway wrappedGateway;
}
  
```

Chaque décorateur concret ajoute une responsabilité précise :

Décorateur	Responsabilité
LoggingPaymentDecorator	Affiche une trace avant et après le paiement
ServiceFeePaymentDecorator	Ajoute des frais de service au coût estimé
FraudCheckPaymentDecorator	Refuse les montants jugés suspects
CashbackPaymentDecorator	Ajoute une information de cashback après paiement réussi
RetryPaymentDecorator	Relance le paiement en cas d'échec

TP4 - Conception avec Décorateur



Source PlantUML : `../diagrams/figure2_decorator_design.puml`

Les boutiques issues du TP3 créent d'abord leurs passerelles, puis appliquent les décorateurs :

```

protected PaymentGateway decorateGateway(PaymentGateway gateway) {
    return new LoggingPaymentDecorator(
        new ServiceFeePaymentDecorator(gateway, 1.50, 0.002)
    );
}
  
```

4. Règles de composition attendues

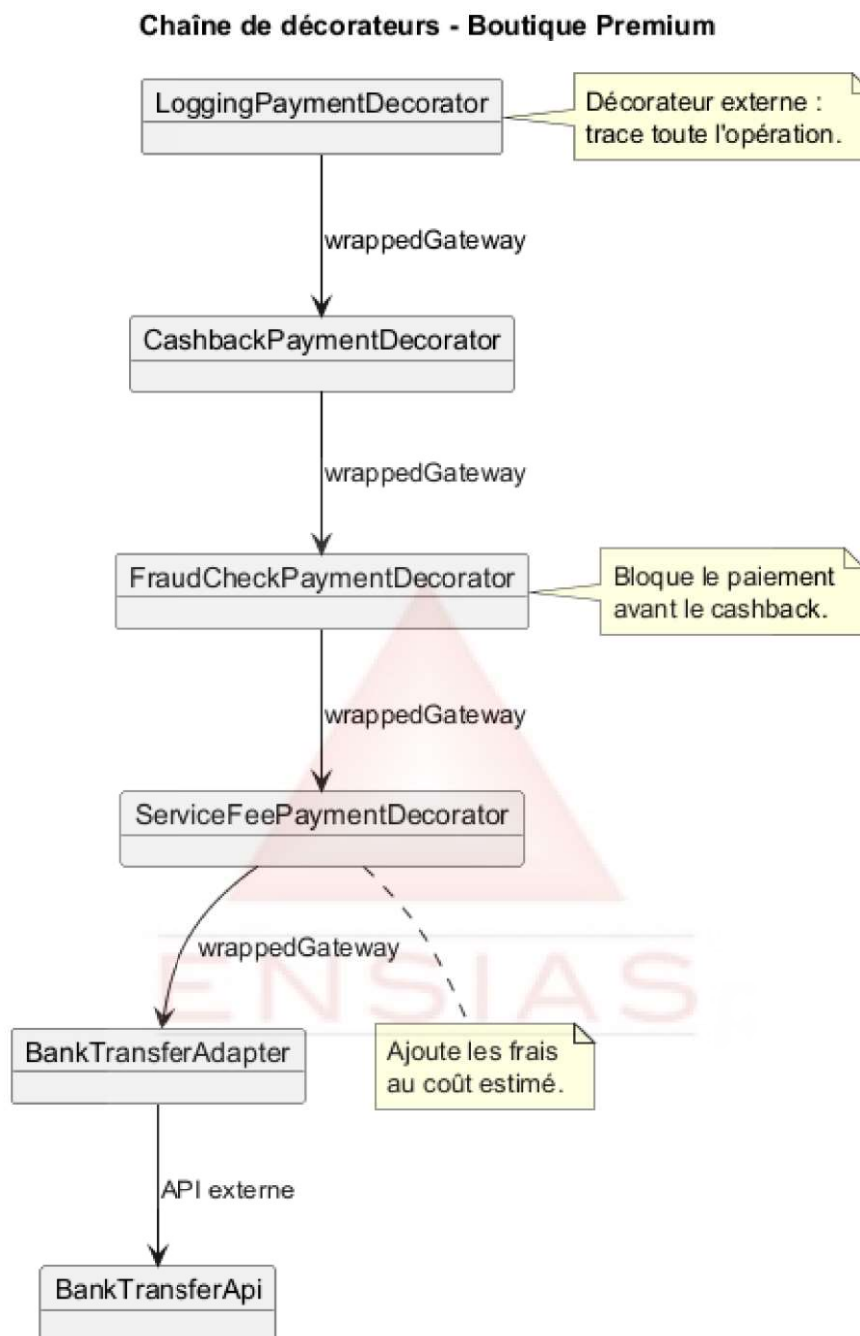
Les règles suivantes sont retenues pour le TP4 :

Boutique	Passerelles	Décorateurs attendus
Maroc	Virement bancaire, PayPal, wallet mobile	Frais de service + logging
Europe	PayPal	Retry + logging
Premium	Virement bancaire, PayPal	Frais de service + anti-fraude + cashback + logging
Afrique	PayPal	Frais de service + retry + logging

L'ordre de composition doit être réfléchi.

Par exemple, dans la boutique Premium, le contrôle anti-fraude doit intervenir avant que le cashback soit annoncé.

Le logging peut être placé à l'extérieur de la chaîne pour tracer l'ensemble de l'opération.



Source PlantUML : `../diagrams/figure6_premium_decorator_chain.puml`

5. Travail demandé

1. Lire le code des packages `adapter/`, `strategy/`, `factory/` et `decorator/`.
2. Identifier les responsabilités déjà couvertes par TP1, TP2 et TP3.
3. Étudier `PaymentGatewayDecorator` et expliquer pourquoi il reste substituable à `PaymentGateway`.
4. Compléter `LoggingPaymentDecorator`.

5. Compléter `ServiceFeePaymentDecorator`.
6. Compléter `FraudCheckPaymentDecorator`.
7. Compléter `CashbackPaymentDecorator`.
8. Vérifier le fonctionnement de `RetryPaymentDecorator`.
9. Composer les décorateurs dans les boutiques concrètes.
10. Lancer `mvn compile`.
11. Lancer `mvn exec:java`.
12. Lancer le test autonome :

```
mvn test-compile exec:java -
Dexec.mainClass=ma.ensias.ecommerce.tp4.DecoratorCompositionTest -
Dexec.classpathScope=test
```

13. Compléter `REPONSES_TP4_A_RENDRE.md`.

6. Contraintes de conception

Respectez les contraintes suivantes :

- ne pas ajouter de logique de logging, frais, anti-fraude, cashback ou retry dans les adaptateurs ;
- ne pas modifier `CheckoutService` pour connaître les décorateurs concrets ;
- conserver l'interface `PaymentGateway` comme contrat unique ;
- éviter les conditions du type `if (premium)`, `if (withLogging)` dans le service de checkout ;
- ajouter chaque responsabilité dans une classe dédiée ;
- garder les stratégies responsables uniquement du choix d'une passerelle.

7. Questions intermédiaires

- Pourquoi les adaptateurs ne doivent-ils pas contenir la logique de logging ou de cashback ?
- Pourquoi un décorateur doit-il implémenter la même interface que l'objet décoré ?
- Pourquoi `CheckoutService` n'a-t-il pas besoin de connaître les décorateurs ?
- Qu'est-ce qui distingue une passerelle créée par `Factory Method` d'une passerelle enrichie par `Decorator` ?
- Que se passe-t-il si l'on change l'ordre des décorateurs ?
- Dans quel cas une composition manuelle des décorateurs devient-elle insuffisante ?

8. Scénarios à observer

Lors de l'exécution de `Main`, observez les scénarios suivants :

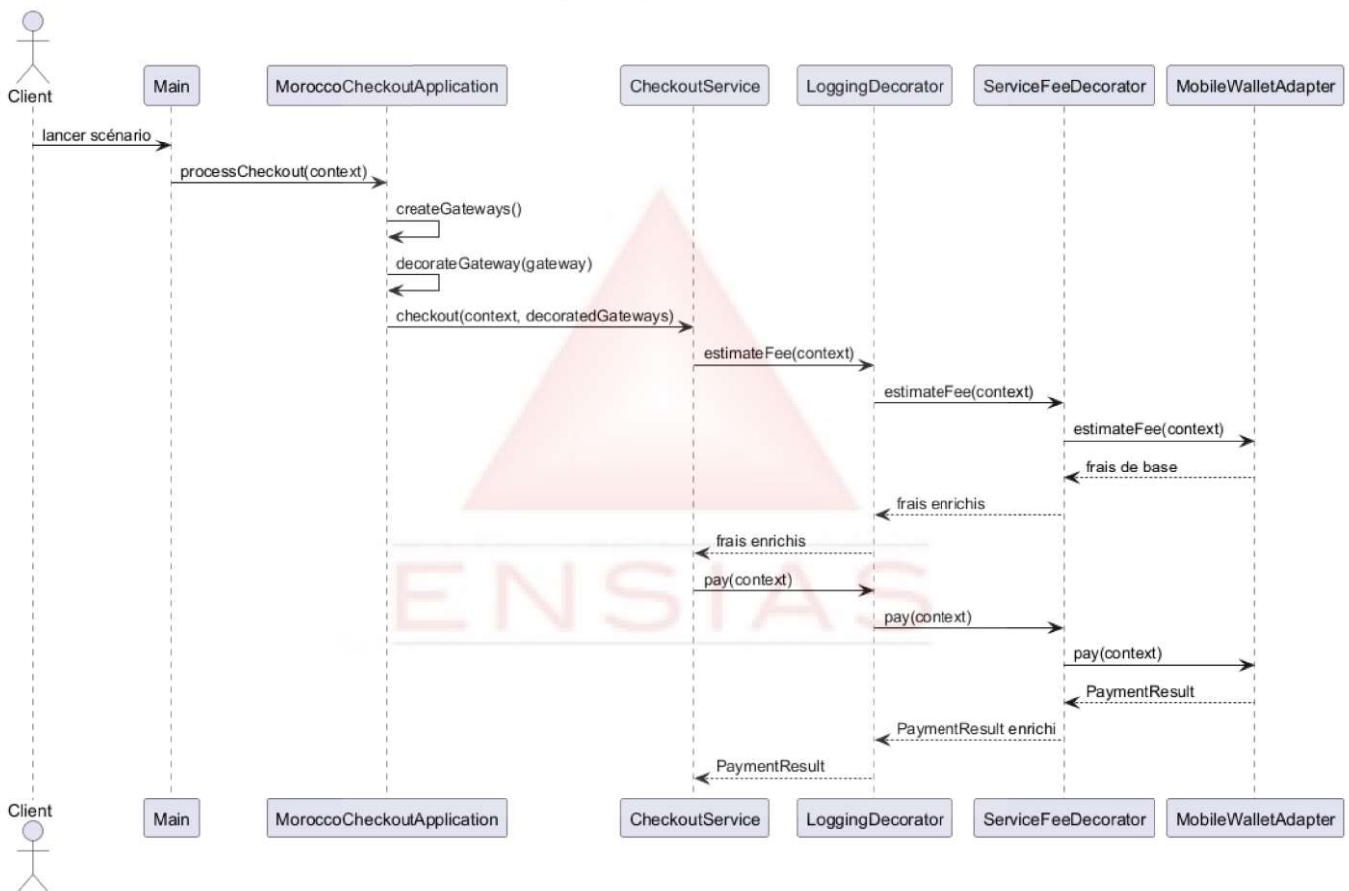
- boutique Maroc avec stratégie de coût minimal ;
- boutique Premium avec stratégie de sécurité maximale ;

- boutique Europe avec stratégie de confirmation rapide ;
- boutique Maroc avec changement de stratégie à l'exécution ;
- boutique Afrique ajoutée sans modifier **CheckoutService** ;
- montant suspect bloqué par le décorateur anti-fraude.

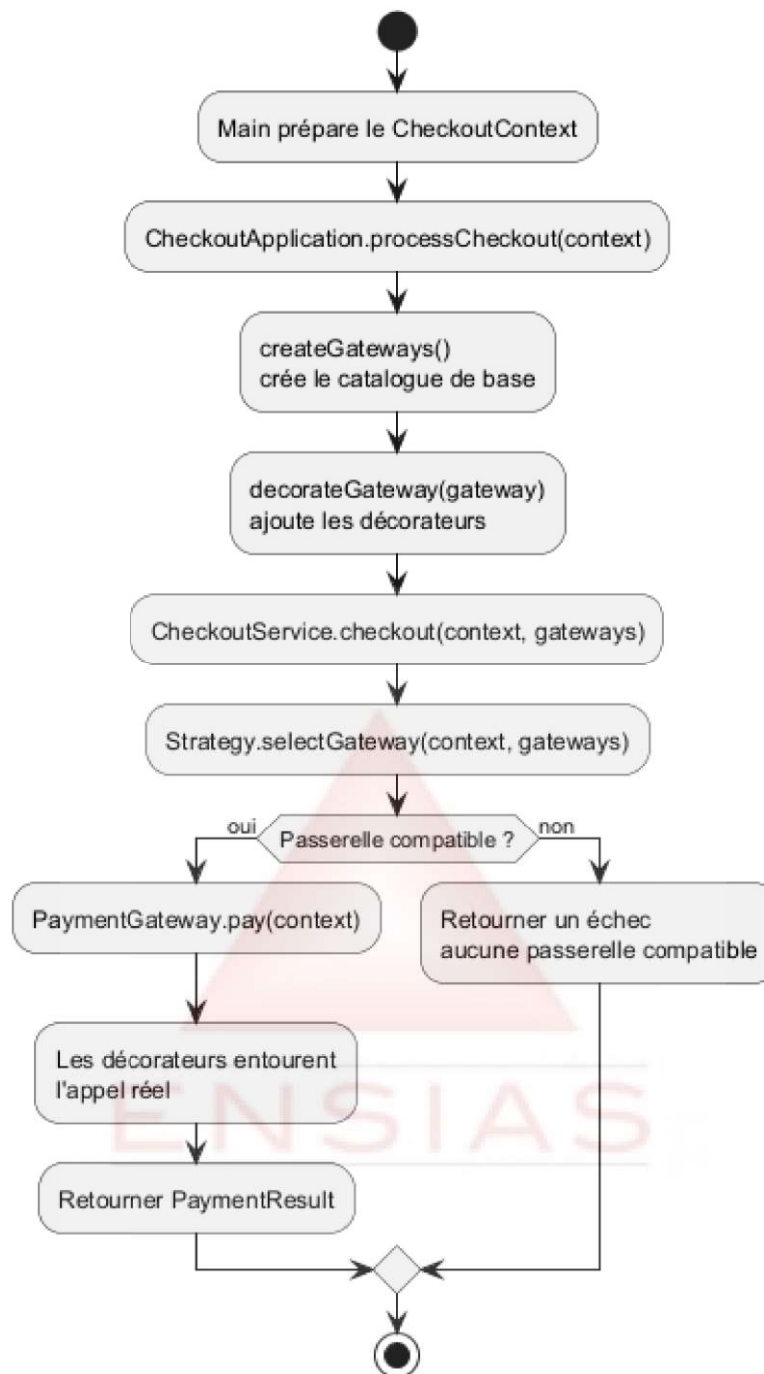
Pour chaque scénario, identifiez :

- la boutique utilisée ;
- la stratégie active ;
- les décorateurs appliqués ;
- la passerelle sélectionnée ;
- le résultat observé.

TP4 - Séquence de paiement avec Décorateur



Source PlantUML : `../diagrams/figure3_decorator_sequence.puml`

Pipeline TP4 : créer, décorer, choisir, payer

Source PlantUML : `../diagrams/figure7_checkout_pipeline.puml`

9. Critère de réussite

Le TP est réussi si :

- le projet compile ;
- les décorateurs ajoutent bien leurs responsabilités ;
- les boutiques composent les décorateurs sans modifier `CheckoutService` ;
- les stratégies continuent à manipuler des `PaymentGateway` ;
- le test `DecoratorCompositionTest` se termine par :

Tous les tests Decorator sont passés.

